



Pandas for Data Science

Day 4 — Lecture Notes & Practical Tasks

GroupBy · Merge · Pivot · Apply · String Ops · Export

1. Sorting & Ranking

Sorting is essential before grouping, visualising, or reporting. Pandas provides fast, flexible sort methods.

1.1 `sort_values` & `sort_index`

```
import pandas as pd, seaborn as sns

df = sns.load_dataset('titanic')

# Single column — ascending (default)
print(df.sort_values('age').head())

# Descending
print(df.sort_values('fare', ascending=False).head())

# Multi-column sort (primary: pclass asc, secondary: age desc)
print(df.sort_values(['pclass', 'age'], ascending=[True,
False]).head(10))

# Sort index
df_shuffled = df.sample(frac=1, random_state=42)
print(df_shuffled.sort_index().head())

# nlargest / nsmallest — quick top-k
print(df.nlargest(5, 'fare')[['who', 'fare', 'pclass']])
print(df.nsmallest(5, 'age')[['who', 'age', 'pclass']])
```

1.2 Ranking

```
import pandas as pd, seaborn as sns

df = sns.load_dataset('titanic').copy()
```

```
# rank() - assigns ranks (1=lowest by default)
df['fare_rank'] = df['fare'].rank(ascending=False, method='dense')
print(df[['who', 'fare', 'fare_rank']].sort_values('fare_rank').head())

# method options:
# 'average' - tied values get average rank
# 'min'      - tied values get minimum rank
# 'max'      - tied values get maximum rank
# 'dense'    - no gaps in rank numbers
# 'first'    - first occurrence gets lower rank

# Percentile rank
df['fare_pct'] = df['fare'].rank(pct=True)
print(df[['fare', 'fare_pct']].head())
```

1.3 Tasks — Sorting & Ranking

Task 1: Sort Titanic by fare descending. Who are the top 10 highest-paying passengers? Print their name (if available), fare, and class.

Task 2: Sort by pclass ascending, then by age descending within each class. Show the youngest in each class.

Task 3: Add a rank column to a 10-student marks DataFrame (rank 1=best). Try all 5 rank methods and compare results for tied students.

2. GroupBy — Split · Apply · Combine

GroupBy is one of Pandas' most powerful features. It follows the split-apply-combine pattern used in SQL, Excel pivot tables, and R's dplyr.

2.1 Basic Aggregation

```
import pandas as pd, seaborn as sns

df = sns.load_dataset('titanic')

# Single groupby + single aggregation
print(df.groupby('pclass')['fare'].mean())
print(df.groupby('sex')['survived'].sum())

# Multiple aggregations on one column
print(df.groupby('pclass')['age'].agg(['mean', 'std', 'count', 'min', 'max']))

# Multiple columns grouped
print(df.groupby(['pclass', 'sex'])['survived'].mean().unstack())

# agg dict - different function per column
result = df.groupby('pclass').agg(
```

```
avg_age    = ('age', 'mean'),
total_fare = ('fare', 'sum'),
survival_pct=('survived','mean'),
passengers = ('survived','count')
)
print(result.round(2))
```

2.2 Transform & Filter

```
import pandas as pd, seaborn as sns

df = sns.load_dataset('titanic').copy()

# transform - returns same-shape Series (no row collapse)
df['class_mean_fare'] = df.groupby('pclass')['fare'].transform('mean')
df['fare_vs_class']   = df['fare'] - df['class_mean_fare'] # deviation

# Z-score normalise within group
df['fare_z'] = df.groupby('pclass')['fare'].transform(
    lambda x: (x - x.mean()) / x.std())

# filter - keep only groups satisfying condition
big_groups = df.groupby('pclass').filter(lambda x: len(x) > 100)
print(big_groups['pclass'].value_counts())

# apply - most flexible (custom function per group)
def top_earners(group, n=3):
    return group.nlargest(n, 'fare')

top = df.groupby('pclass').apply(top_earners)
print(top[['pclass', 'who', 'fare']])
```

2.3 Tasks — GroupBy

Task 1: On Titanic: compute survival rate by sex, by pclass, and by (sex × pclass) combination. Which group had the highest survival rate?

Task 2: For each passenger class, find the mean, median, and std of fares. Use a single .agg() call.

Task 3: Add a column class_median_age = median age within each pclass using transform. Then add a flag IsOlderThanClassMedian.

Task 4: Filter out passenger classes with fewer than 200 passengers. How many passengers remain?

Task 5: Group by embarked port and compute: total passengers, survival rate, average fare. Sort by survival rate descending.

3. Merging & Joining DataFrames

3.1 pd.merge — SQL-style Joins

```
import pandas as pd

students = pd.DataFrame({
    'id': [1,2,3,4],
    'name': ['Alice', 'Bob', 'Carol', 'Dave'],
    'dept': ['CS', 'Math', 'CS', 'Phy'],
})

scores = pd.DataFrame({
    'id': [1,2,3,5],
    'score': [88, 75, 92, 65],
})

inner = pd.merge(students, scores, on='id', how='inner') # 3 rows
left = pd.merge(students, scores, on='id', how='left') # 4 rows, Dave
NaN
right = pd.merge(students, scores, on='id', how='right') # 4 rows, id=5
NaN
outer = pd.merge(students, scores, on='id', how='outer') # 5 rows

print(left)

# Different column names
scores2 = scores.rename(columns={'id': 'student_id'})
merged = pd.merge(students, scores2,
                  left_on='id', right_on='student_id')
print(merged)
```

3.2 pd.concat

```
import pandas as pd

df1 = pd.DataFrame({'A': [1,2], 'B': ['x', 'y']})
df2 = pd.DataFrame({'A': [3,4], 'B': ['z', 'w']})
df3 = pd.DataFrame({'C': [10,20], 'D': [30,40]})

# Stack rows
vertical = pd.concat([df1, df2], ignore_index=True)
print(vertical)

# Stack columns
horizontal = pd.concat([df1, df3], axis=1)
print(horizontal)

# With keys (creates MultiIndex)
labeled = pd.concat([df1, df2], keys=['batch1', 'batch2'])
print(labeled)
print(labeled.loc['batch1'])
```

3.3 Tasks — Merge & Concat

Task 1: Create a Students table (id, name, age) and a Grades table (id, subject, marks). Perform all 4 join types and explain the row count difference.

Task 2: You have two monthly sales DataFrames (same columns). Concatenate them into a single yearly DataFrame. Add a Month column before concatenating.

Task 3: Merge Titanic with a separate DataFrame mapping pclass → class_label ('First','Second','Third'). Verify all rows have a label.

Task 4: Detect duplicate rows after a merge. Use df.duplicated() to find and df.drop_duplicates() to remove them.

4. Pivot Tables & Crosstab

4.1 pivot_table

```
import pandas as pd, seaborn as sns

df = sns.load_dataset('titanic')

# Average fare by class and sex
pt = pd.pivot_table(df,
                    values='fare', index='pclass', columns='sex',
                    aggfunc='mean', margins=True)
print(pt.round(2))

# Survival rate with count fill
pt2 = pd.pivot_table(df,
                    values='survived', index='pclass', columns='sex',
                    aggfunc=['mean', 'count'])
print(pt2)

# Multiple values
pt3 = pd.pivot_table(df,
                    values=['fare', 'age', 'survived'],
                    index='pclass', aggfunc='mean')
print(pt3.round(2))
```

4.2 crosstab

```
import pandas as pd, seaborn as sns

df = sns.load_dataset('titanic')

# Frequency count
ct = pd.crosstab(df['pclass'], df['survived'])
print(ct)

# Column percentages (how many in each class survived?)
ct_pct = pd.crosstab(df['pclass'], df['survived'],
```

```
        normalize='index') * 100
print(ct_pct.round(1))

# Add margins
ct_m = pd.crosstab(df['sex'], df['survived'], margins=True)
print(ct_m)

# Three-way crosstab
ct3 = pd.crosstab([df['sex'], df['pclass']], df['survived'])
print(ct3)
```

4.3 Tasks — Pivot & Crosstab

Task 1: Build a pivot table: mean and std of age grouped by pclass (rows) and survived (columns). Include totals.

Task 2: Use crosstab to show percentage survival by sex AND embarked port. Which combination had the highest survival rate?

Task 3: Pivot Titanic to show total fare collected per class per embarkation port. Which port generated the most revenue in each class?

5. Apply, Map & Custom Transformations

5.1 map, apply, applymap

```
import pandas as pd, numpy as np, seaborn as sns

df = sns.load_dataset('titanic').copy()

# map — Series element-wise (dict replacement)
df['sex_code'] = df['sex'].map({'male':0, 'female':1})

# apply — Series, custom function
df['age_sq'] = df['age'].apply(lambda x: x**2 if pd.notna(x) else
np.nan)

# apply — DataFrame row-wise (axis=1)
def classify(row):
    if row['survived']==1 and row['age'] < 18:
        return 'Saved Child'
    elif row['survived']==1:
        return 'Saved Adult'
    else:
        return 'Not Saved'

df['status'] = df.apply(classify, axis=1)
print(df['status'].value_counts())
```

```
# applymap (deprecated in newer pandas: use map instead)
nums = df.select_dtypes('number')
rounded = nums.applymap(lambda x: round(x,1) if pd.notna(x) else x)

# pd.cut — binning
df['fare_band'] = pd.cut(df['fare'],
    bins=[0,10,50,200,600],
    labels=['Budget','Economy','Premium','Luxury'])
```

5.2 Tasks — Apply & Map

Task 1: Use map to replace Titanic 'embarked' codes (C/Q/S) with full names (Cherbourg/Queenstown/Southampton).

Task 2: Use apply (axis=1) to create a column 'Outcome': 'Survived-Rich' (survived & fare>50), 'Survived-Poor', 'Died-Rich', 'Died-Poor'.

Task 3: Use pd.cut to bin ages into 5 equal-width bands. Then use pd.qcut for 5 quantile-based bands. Compare the distribution.

Task 4: Without using loops, use applymap (or df.map in newer pandas) to round every numeric cell in a DataFrame to 2 decimal places.

6. String Operations — str Accessor

```
import pandas as pd

df = pd.DataFrame({
    'Name': ['Alice Smith', 'BOB Jones ', 'carol LEE'],
    'Email': ['alice@mail.com', 'bob@work.org', 'carol@mail.com'],
    'City': ['New York-NY', 'Los Angeles-CA', 'Chicago-IL'],
})

# Clean
df['Name'] = df['Name'].str.strip().str.title()
df['Email'] = df['Email'].str.lower()

# Split
df[['City_Name', 'State']] = df['City'].str.split('-', expand=True)

# Filter
mail_users = df[df['Email'].str.contains('@mail')]
print(mail_users)

# Length
df['name_len'] = df['Name'].str.len()

# Replace
df['City'] = df['City'].str.replace('-', ' - ')
```

```
# Regex extract (last word of name)
df['Surname'] = df['Name'].str.extract(r'(\w+)$')
print(df)
```

6.1 Tasks — String Operations

Task 1: Load a CSV with messy names (mixed case, extra spaces). Clean: strip, title-case, extract first and last name into separate columns.

Task 2: From a product code column like 'AB-1234-XY', extract: category (first 2 chars), ID (middle 4 digits), suffix (last 2 chars).

Task 3: From Titanic 'who' column, use str.contains to find all passengers whose name contains 'Mr' or 'Mrs'. Count each group.

Task 4: From an email column, extract the domain (e.g., 'gmail' from 'user@gmail.com') using str.extract with a regex group.

7. Exporting & Full EDA Workflow

7.1 Export Functions

```
import pandas as pd

df = pd.DataFrame({'A':[1,2,3], 'B':['x','y','z'], 'C':[1.1,2.2,3.3]})

# CSV
df.to_csv('output.csv', index=False)

# Excel (pip install openpyxl)
df.to_excel('output.xlsx', sheet_name='Results', index=False)

# JSON
df.to_json('output.json', orient='records', indent=2)

# Parquet (pip install pyarrow) — fastest for large data
df.to_parquet('output.parquet', index=False)

# Verify round-trip
df2 = pd.read_csv('output.csv')
print(df.equals(df2))    # True

# Export multiple sheets to Excel
with pd.ExcelWriter('multi_sheet.xlsx') as writer:
    df.to_excel(writer, sheet_name='Data', index=False)
    df.describe().to_excel(writer, sheet_name='Stats')
```

7.2 Tasks — Export

Task 1: Complete Titanic EDA: clean → feature engineer → groupby analysis → export clean CSV and an Excel with 2 sheets (Data + Stats).

Task 2: Save three different subsets of Titanic (by pclass) to separate CSV files programmatically using a loop.

8. Comprehensive Challenges — Day 4

These challenges combine all Pandas concepts from both days. Use real datasets where possible.

8.1 Sales Analytics Pipeline

Create a 50-row sales DataFrame: Date, Salesperson, Region, Product, Units, UnitPrice. (a) Add Revenue = Units × UnitPrice. (b) GroupBy Region → total revenue, units sold, average deal size. (c) Pivot: Region (rows) vs Product (cols) → total revenue. (d) Rank salespersons by total revenue. (e) Export: clean CSV + Excel with 3 sheets (raw, summary, pivot).

8.2 Titanic Full Analysis

Start with raw Titanic data (seaborn). (a) Clean all missing values with justified strategies. (b) Add: FamilySize, IsAlone, AgeGroup, FareBand (pd.qcut, 4 bins). (c) Crosstab: survival by AgeGroup × Sex. (d) GroupBy pclass: mean age, mean fare, survival rate, passenger count. (e) Merge with a class_info DataFrame (pclass → label, deck_level). (f) Export: titanic_final.csv + pivot table to Excel.

8.3 Text Data Cleaning

Download any CSV with a text column (e.g., product descriptions, customer names). (a) Strip whitespace, fix casing. (b) Extract tokens using str.split. (c) Flag rows containing specific keywords with str.contains. (d) Extract structured data from strings using str.extract with regex. (e) Count word frequency using Series.value_counts after str.split explosion.

8.4 Monte Carlo with Pandas

Simulate 10,000 game rounds: each round roll 2 dice (sum = score). Store in a DataFrame with columns: round, die1, die2, total. (a) GroupBy total → frequency and probability. (b) Find probability of getting a sum > 9. (c) Use pivot_table to show frequency of (die1, die2) combinations. (d) Export results to Excel with a stats sheet.